



Operating Systems

Unit 2

Subject Code: CI42

B.E Sem: IV

Credits : 4



Syllabus

Unit II

Process Management: Basic concept; Process scheduling; Operations on processes; Inter process Communication.

Threads: Overview; Multithreading models;

Process scheduling: Basic concepts, Scheduling criteria, scheduling algorithms, multiple processor scheduling, Algorithm evaluation.



Definition: A thread is a fundamental, basic unit of CPU utilization.

Anatomy of a Thread: Every individual thread comprises four unique things:

1. A Thread ID
2. A Program Counter
3. A Register Set
4. A Stack

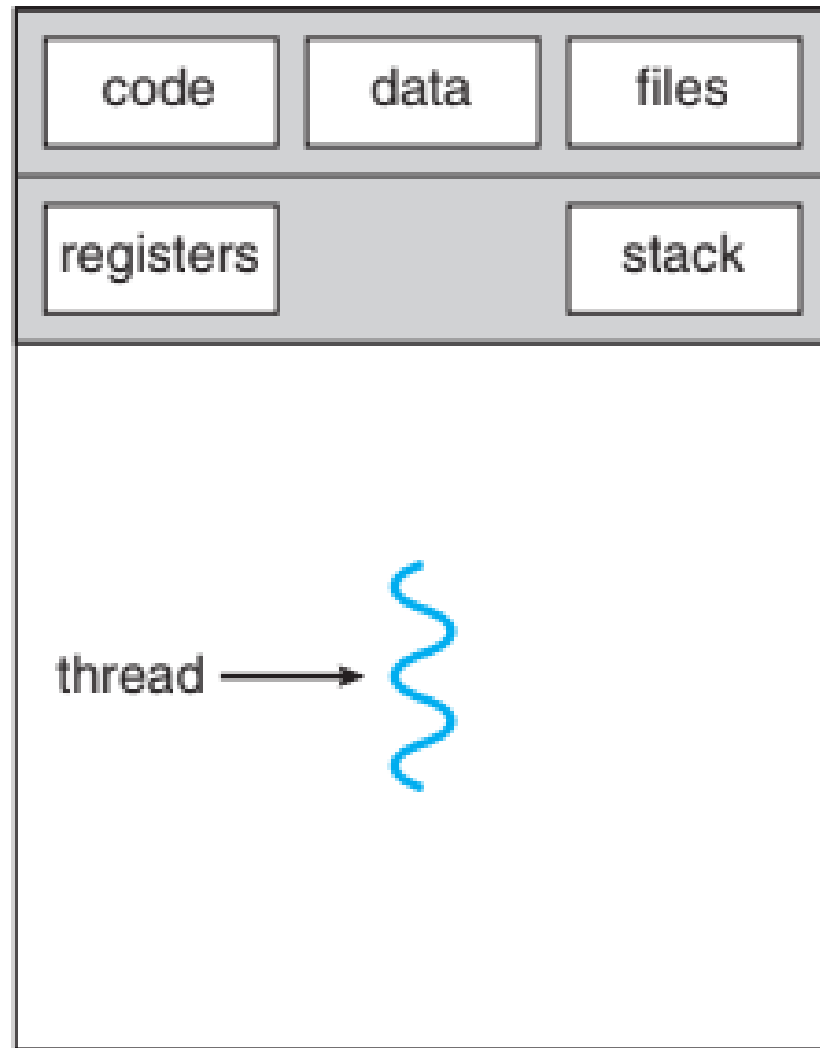


What is a Thread?

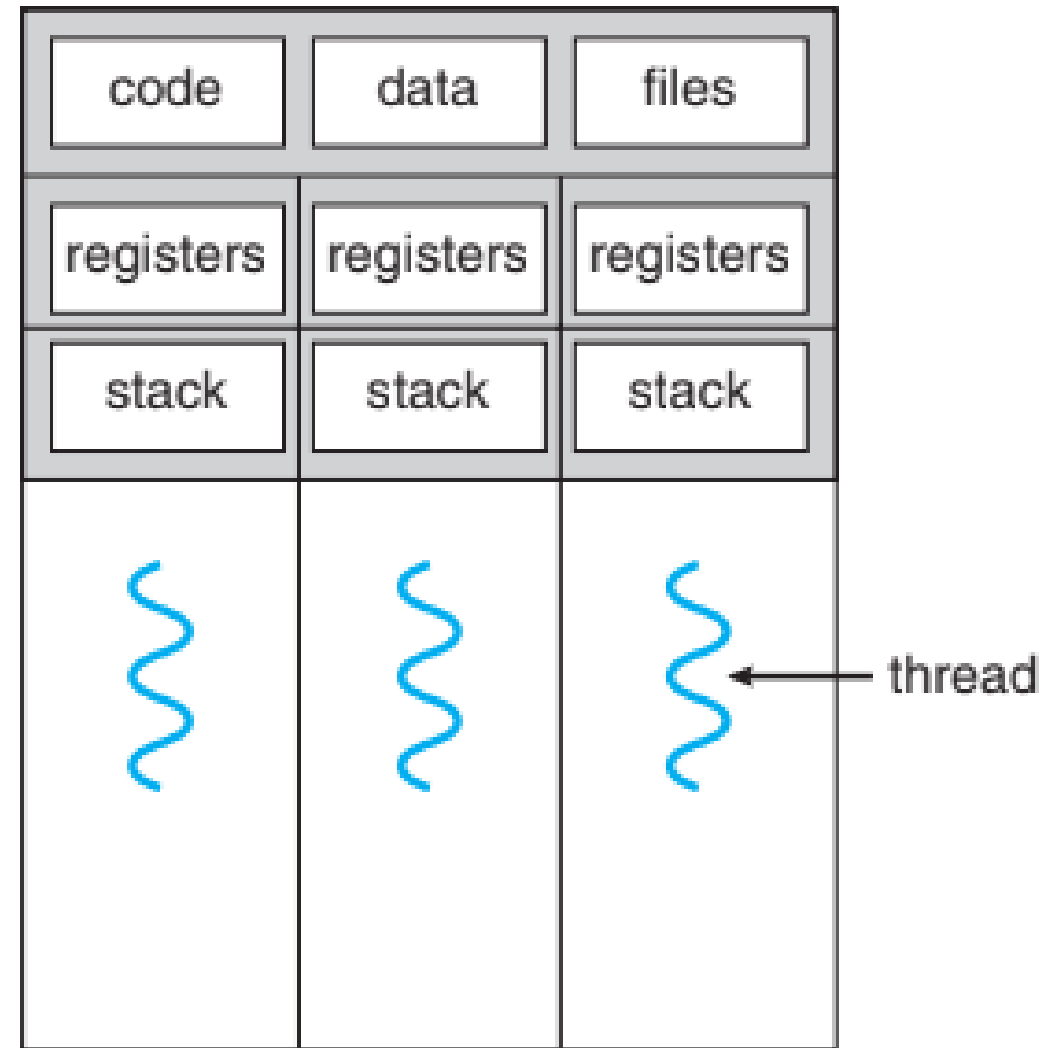
What is Shared? Threads belonging to the same process share the process's:

- Code section (text)
- Data section
- Other OS resources (like open files and signals).

Heavyweight vs. Multithreaded: A traditional process is "heavyweight" and has only a single thread of control. A multithreaded process has multiple threads of control, allowing it to perform more than one task at exactly the same time.



single-threaded process



multithreaded process

Figure 4.1 Single-threaded and multithreaded processes.



Motivation for Multithreading

The Modern Standard: Most software applications that run on modern computers are multithreaded.

Everyday Examples:

- **Web Browser:** One thread displays images and text, while completely separate thread retrieves data from the network in the background.
- **Word Processor:** One thread displays graphics, a second thread responds to your keystrokes, and a third thread performs spelling/grammar checking simultaneously.

The Multicore Advantage: Applications can be designed to leverage multiple computing cores. A multithreaded application can perform several CPU-intensive tasks in parallel by distributing its threads physically across different cores.

Multithreaded Server Architecture & OS Kernels

The Web Server Problem: A busy web server might have thousands of clients concurrently accessing it.

The Old Solution (Processes): The server receives a request and creates a brand-new separate process to service it. Problem: Process creation is incredibly time-consuming and resource-intensive.

The Modern Solution (Threads): The server runs as a single process with multiple threads.

- One thread simply listens for client requests.
- When a request arrives, the server creates a new thread to service the request, and the listening thread quickly resumes listening for more clients.

Multithreaded Server Architecture & OS Kernels

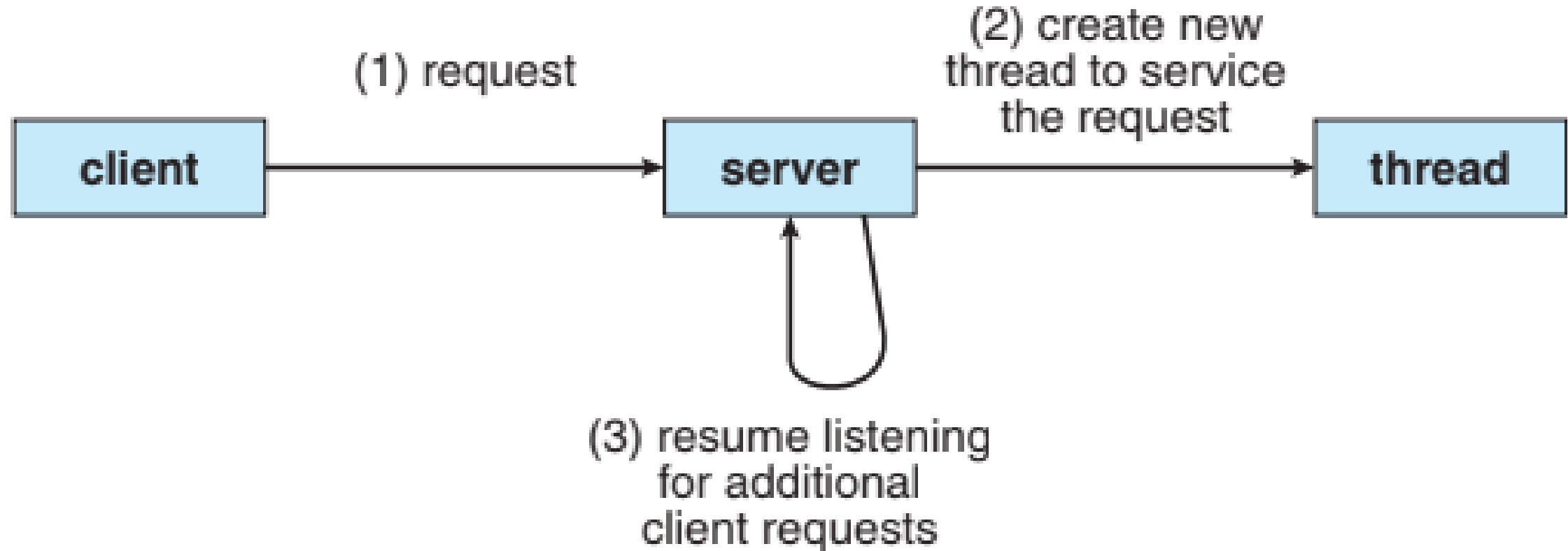


Figure 4.2 Multithreaded server architecture.

The 4 Major Benefits of Multithreading

1. Responsiveness:

- Allows an interactive application to continue running even if a part of it is blocked or performing a lengthy operation.
- Example: If a time-consuming operation (like a large download) is in a separate thread, the user interface remains responsive to the user.

2. Resource Sharing:

- Unlike processes (which require explicit shared memory or message passing), threads share the memory and resources of their parent process by default.
- Allows an application to have several different threads of activity perfectly cooperating within exact same address space.



3. Economy:

- Allocating memory and resources for brand-new process creation is highly costly.
- It is significantly more economical and faster to create and context-switch threads.
- Example: In Solaris, creating a process is about 30 times slower than creating a thread, and context switching is about 5 times slower.

4. Scalability:

- The benefits are vastly increased in a multiprocessor (or multicore) architecture.
- A single-threaded process can only run on one processor, no matter how many are available. Multithreading allows threads to run in true parallel across different processing cores.



User Threads vs. Kernel Threads

Up to this point, we have treated threads generically. However, support for threads is provided at two distinct levels:

User Threads:

- Supported above the kernel.
- Managed completely by a thread library in user space, meaning they are managed without kernel support.

Kernel Threads:

- Supported and managed directly by the operating system kernel.
- Virtually all contemporary operating systems (Windows, Linux, Mac OS X, Solaris) support kernel threads.

The Bridge: Ultimately, for a program to actually run on the CPU, a relationship must be established between the user threads (created by the programmer) and the kernel threads (managed by the OS).



The Many-to-One Model

The Mapping: The Many-to-One model maps many user-level threads to one single kernel thread.

Advantages: Thread management is done by the thread library in user space, which makes it highly efficient.

Major Drawbacks:

- **Blocking:** If a single user thread makes a blocking system call (like waiting for I/O), the single kernel thread halts, meaning the entire process will block.
- **No Parallelism:** Because only one thread can access the kernel at a time, multiple threads are entirely unable to run in parallel on multicore systems.

Usage: Very few modern systems continue to use this model because it cannot take advantage of multiple processing cores. (Examples: Solaris Green threads, early Java).



The Many-to-One Model

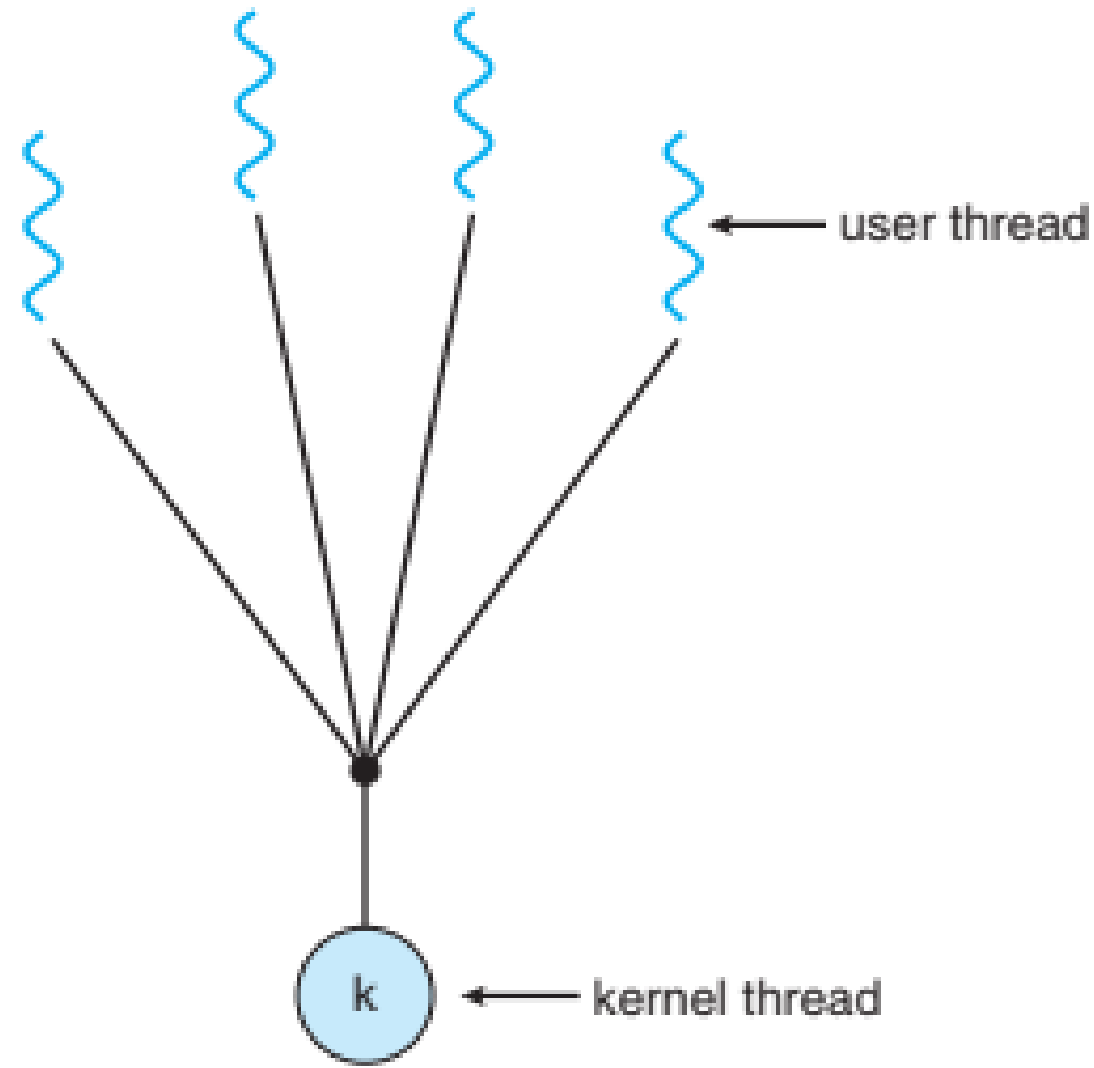


Figure 4.5 Many-to-one model.

Mechanism: Maps each individual user thread to its own single kernel thread.

Advantages over Many-to-One:

- Provides significantly more concurrency. It allows another thread to run when a thread makes a blocking system call.
- Allows multiple threads to run in true parallel on multiprocessors.



The Major Drawback:

- Creating a user thread strictly requires creating the corresponding kernel thread.
- The overhead of creating kernel threads can burden the performance of an application.
- **Solution:** Most implementations restrict the maximum number of threads supported by the system.

Implementations: Linux, and the family of Windows operating systems, implement the one-to-one model.

The One-to-One Model

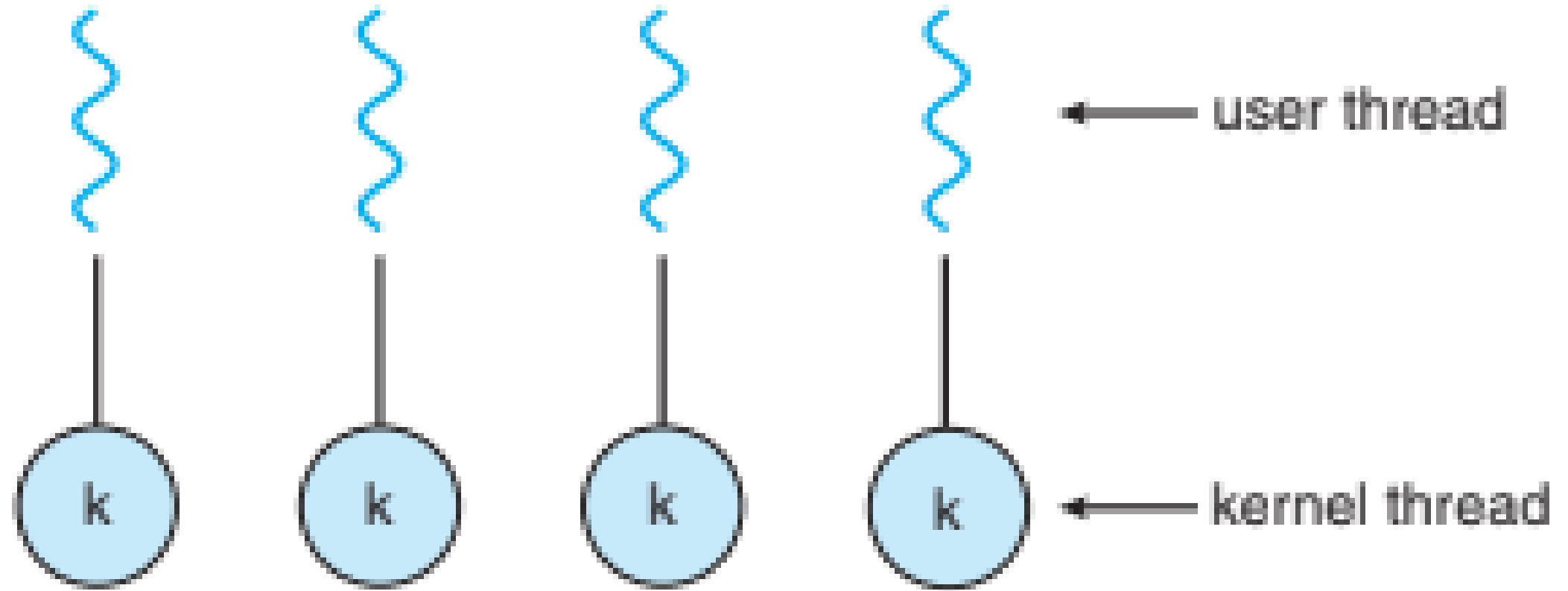


Figure 4.6 One-to-one model.



The Many-to-Many Model:

- Multiplexes many user-level threads to a smaller or equal number of kernel threads.
- The number of kernel threads dynamically adjusts based on the specific application or machine (e.g., allocating more on a multiprocessor).

The Best of Both Worlds:

- Developers can create as many user threads as necessary (unlike One-to-One).
- Kernel threads can run in parallel on multiprocessors, and blocking calls don't block the whole process (unlike Many-to-One).



The Two-Level Model:

- A variation of the Many-to-Many model.
- It multiplexes many user threads to kernel threads, but also allows a specific user-level thread to be strictly bound to a dedicated kernel thread if needed.
- Examples: IRIX, HP-UX, Tru64 UNIX, and Solaris (prior to version 9).

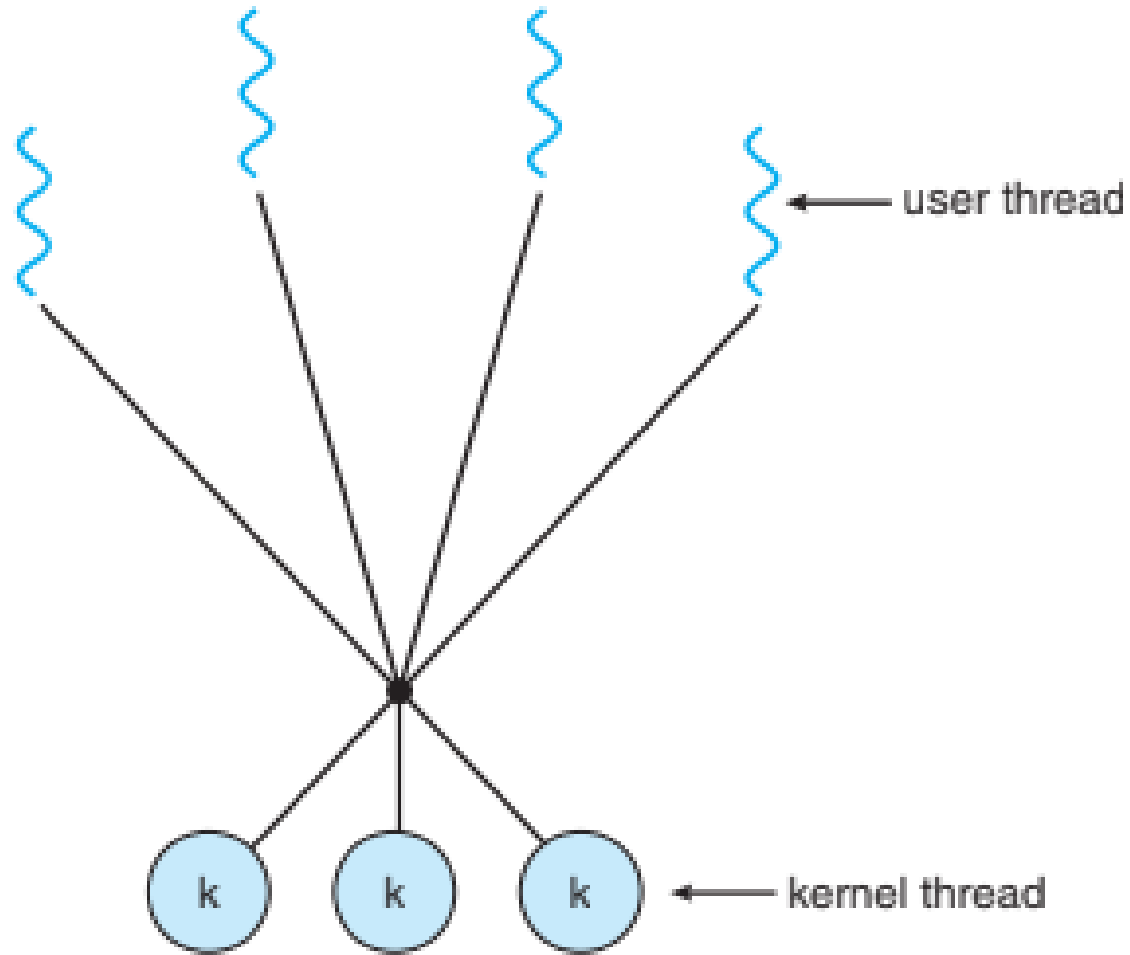


Figure 4.7 Many-to-many model.

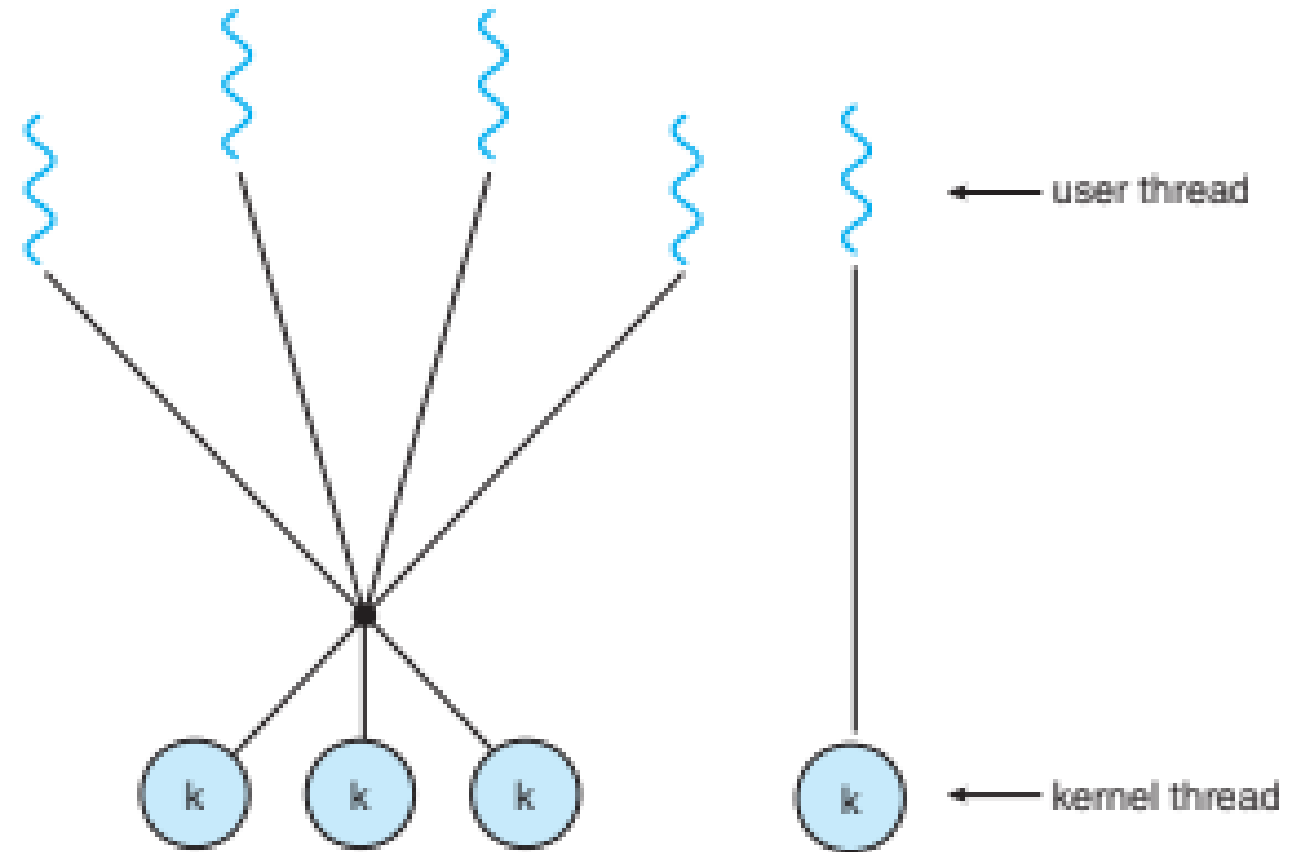


Figure 4.8 Two-level model.